

Práctica Parser

1. Escribir un trasformador que al recibir un parser, devuelve un nuevo parser que se comporta como el original pero que también acepta opcionalmente las cadenas que estén entre paréntesis

2. Escribir un trasformador de parsers *withQuotes* tal que, dado un parser p, el parser resultante acepte opcionalmente el mismo contenido encerrado entre comillas simples o dobles. Por ejemplo, el parser *withQuotes* (*string* `"hola"`) debe aceptar tanto *hola* como `'hola'` y `"hola"`.

3. Definir un parser *sepList* que reciba un parser de elementos y una lista de parsers de separadores y que acepte listas donde los elementos pueden estar separados por cualquiera de ellos. Por ejemplo, *sepList int* [*char* `','`, *char* `';'`, *string* `" o "`] debe reconocer:

- $1, 2, 3$
- $4; 5; 6$
- 7 *o* 8 *o* 9

4. Definir un parser que reconozca tuplas anidadas de números enteros, como:

$$(1, 2)\ ((3, 4), (5, 6))\ (((1, 2), 3), 4)$$

El parser debe construir una estructura de tipo:

data *Tuple* = *Single Int* | *Pair Tuple Tuple*

5. Implementar un parser *interval* que reconozca intervalos de la forma:

$$(a, b),\ [a, b),\ (a, b]\ \text{o}\ [a, b].$$

y genere un valor de tipo:

data *Interval* = *Interval* { *leftOpen* :: *Bool*
, *rightOpen* :: *Bool*
, *a* :: *Float*
, *b* :: *Float* }

El parser debe interpretar los paréntesis y corchetes como indicadores de apertura y cierre de los extremos.

6. Escribir un parser para vectores linealmente espaciados, con la siguiente sintaxis: *linspace* $(inicio, fin, n)$

- *inicio* y *fin* son las cotas para generar los valores del vector
- n es un valor entero que determina la cantidad de valores pedidos.

Ejemplo:*linspace* $(1, 3, 6) = [1.0, 1.4, 1.8, 2.2, 2.6, 3.0]$

7. Escribir un parser para listas heterogéneas de enteros y caracteres. Definir un tipo de datos en haskell adecuado para representarlas. Ejemplo $[3, \texttt{'z'}, \texttt{'r'}, 7, 9, 22, \texttt{'1'}]$

8. Modificar el parser de expresiones visto en clase para usar también la resta y la división, basándose en la siguiente gramática:

$expr \rightarrow term$ (`'+'` $expr$ | `'-'` $expr$ | $emptyS$)
$term \rightarrow factor$ (`'*'` $term$ | `'/'` $term$ | $emptyS$)

y para que en lugar de evaluar una expresión genere un árbol de sintaxis abstracta dado por el tipo:

data *Expr* = *Num Int* | *BinOp Op Expr Expr*
data *Op* = *Add* | *Mul* | *Res* | *Div*

9. Eliminar recursión izquierda e implementar el parser para la gramática resultante.

$expr \rightarrow expr$ (`'+'` $term$ | `'-'` $term$) | $term$
$term \rightarrow term$ (`'*'` $factor$ | `'/'` $factor$) | $factor$
$factor \rightarrow digit$ | `'('` $expr$ `')'`
$digit \rightarrow$ `'0'` | `'1'` | `'2'` | ... | `'9'`

10. Implementar un parser que reconozca matrices definidas con la sintaxis de Scilab/MATLAB, donde los elementos de cada fila se separan con comas o espacios y las filas se separan con punto y coma. Ejemplo:

$$[1\ 2\ 3; 4\ 5\ 6; 7\ 8\ 9].$$

Debe producir una lista de listas de números:

$$[[1, 2, 3], [4, 5, 6], [7, 8, 9]].$$

11. Implementar un parser para expresiones booleanas con los operadores lógicos *and*, *or*, *not* y paréntesis. Ejemplo:

$$not\ (True\ or\ False\ and\ True).$$

Debe generar un AST:

data *BoolExpr* = *BVal Bool*
| *Not BoolExpr*
| *And BoolExpr BoolExpr*
| *Or BoolExpr BoolExpr*